DATA STRUCTURES & ALGORITHMS

The Master Revision Handbook: Visual Patterns, Invariants & Production Code

A Concise, Visual, and Mechanism-Focused Interview & Revision Handbook

METADATA FIELD	SPECIFICATION
Subject:	Complete DSA Patterns, Invariants & Rigorous Python 3 Code
Target Audience:	SDE / Technical Interviews (Top Tech, FAANG, Product Firms)
Methodology:	Internal Mechanics • State Transitions • Verified Code Recipes • Trade-offs
Scope & Focus:	Complete DSA Patterns, Invariants & Rigorous Python 3 Code
Source Basis:	Synthesized from Comprehensive, Complete & Final Revision Handbooks
Architecture:	20 Core Chapters • 65+ Battle-Tested Interview Boilerplates
Target Roles:	Software Engineer • Machine Learning Engineer • Systems Architect

Visual Diagram: Universal 5-Step Algorithmic Reasoning Model

```
THE UNIVERSAL PROBLEM-SOLVING WORKFLOW:
  [Problem Statement]
          |
          v
  1. INPUT CONSTRAINTS & RANGE   ---> Identify Big-O bounds (e.g., N=10^5 => O(N) or O(N log N))
          |
          v
  2. PATTERN IDENTIFICATION      ---> Monotonicity? Dynamic window? Overlapping subproblems?
          |
          v
  3. STATE & INVARIANT DEFINITION ---> What must strictly hold true before and after each loop iteration?
          |
          v
  4. STEP-BY-STEP TRANSITION     ---> Two Pointers advance, Monotonic Stack pop, DP recurrence
          |
          v
  5. TERMINATION & BASE CASES    ---> Boundary checks: empty inputs, single element, negative keys
```

CORE DESIGN PHILOSOPHY

This master handbook merges all three prior DSA revision editions into an unified, uncompromising reference. Every chapter features structural workflows, visual invariant diagrams, complexity badges, and copy-paste-ready Python 3 templates. Review the patterns actively: dry-run these templates until they flow automatically under interview pressure.

A structured architectural map of all 20 chapters across the revision handbook.

Ch.	Chapter Title & Core Scope	Target Archetypes & Key Problems
1	Complexity & Python Memory	Big-O rules, reference model, shallow vs deep copy, list growth, recursion scope
2	Arrays, Strings & Hashing	Anagram signatures, in-place 2D rotation, product except self, collision mechanics
3	Two Pointers (4 Archetypes)	Converging, read/write, center expansion, 2-list merge, Two Sum II, Valid Palindrome II
4	Fast & Slow (Floyd's Algorithm)	Mathematical cycle proof (L = kC - d), cycle entrance (LC 142), duplicate number (LC 287)
5	Sliding Window (Fixed & Dynamic)	LC 643 max average, LC 3 longest substring, LC 209 min size subarray, LC 424 replacement
6	Monotonic Deque & Window Max	LC 239 Sliding Window Maximum, monotonic decreasing deque invariant, O(N) rolling max
7	Prefix Sums & Range Manipulation	Subarray sum equals K, negative numbers non-monotonicity, 2D prefix matrix, difference arrays
8	Stacks & Monotonic Stacks	Next Greater Element, Daily Temperatures, MinStack O(1), valid parentheses matching
9	Queues & Deque Architecture	collections.deque chunked memory vs list O(N), LC 933 recent counter, LC 232 2-stack queue
10	Binary Search & Search-on-Answer	Closed vs half-open intervals, lower bound, 2D matrix virtual 1D, Koko bananas (LC 875)
11	Linked Lists Manipulation	In-place reversal, sentinel dummy nodes, merge sorted lists, 3-stage reorder list (LC 143)
12	Trees & Binary Search Trees	BFS level order batching, balanced tree bottom-up O(N), validate BST bounds, LCA, LC 105
13	Heaps & Priority Queues	heapify O(N) proof vs O(N log N), top-K elements, dual-heap median finder, interval pairing
14	Recursion & Backtracking	Choose-explore-unchoose, subsets (i+1) vs combinations (i) vs permutations, Catalan parentheses
15	Graphs & Grid Algorithms	Number of islands BFS, Kahn's topological sort, Dijkstra min-heap, Disjoint Set Union (DSU)
16	Dynamic Programming (1D & 2D)	Kadane's streak, Coin Change, House Robber, 0/1 Knapsack reverse loop, LIS O(N log N), LCS
17	Greedy Algorithms & Intervals	Jump Game I & II reachability, Merge Intervals, Non-overlapping intervals, Gas Station circuit
18	Bitwise, Tries & Number Theory	Trie prefix tree, Kernighan's bit trick, power of two, XOR single number, Dutch Flag, Prime Sieve
19	Pattern Decision Guide & Traps	Input size N vs allowable Big-O, 'False Friends' table debunking traps, 5-phase execution loop
20	Master Boilerplates & Complexity	6 core reusable Python templates, full operations complexity matrix, 60-second sanity checklist

CHAPTER 1 - COMPLEXITY ANALYSIS & PYTHON MEMORY MECHANICS

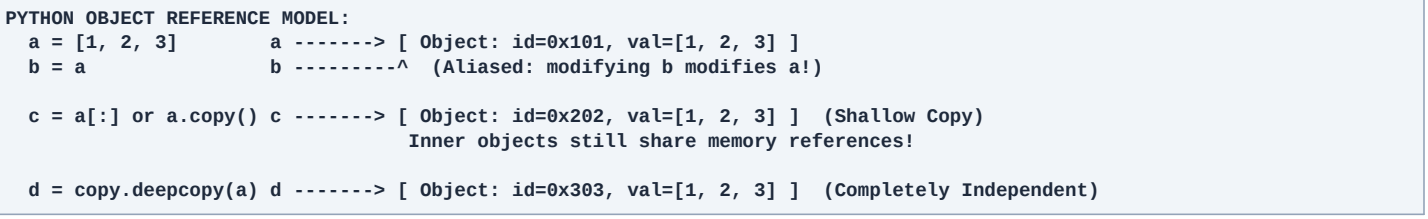
Optimal algorithmic design requires a deep understanding of Big-O limits, Python's object reference model, and the hidden costs of dynamic data structures.

1.1 Big-O, Big-Omega, Big-Theta & Asymptotic Growth Hierarchy

Notation	Formal Definition	Input Boundary (1s Limit ~ 10^7 ops)
O(1)	Constant: independent of input size N	Practically unlimited (Direct arithmetic, Hash Map lookup)
O(log N)	Logarithmic: search space halved each step	N <= 10^18 (Binary Search, Balanced BST, Exponentiation)
O(N)	Linear: single pass through data	N <= 10^7 (Two Pointers, Sliding Window, Kadane's)
O(N log N)	Linearithmic: divide-and-conquer	N <= 10^6 (TimSort, MergeSort, HeapSort, Priority Queue)
O(N^2)	Quadratic: nested loops over input	N <= 5,000 (Bubble Sort, 2D Grid DP, Pairwise comparisons)
O(2^N)	Exponential: 2 choices per state	N <= 20-25 (Subsets, Combinations, Recursion tree)
O(N!)	Factorial: all permutations	N <= 10-12 (Traveling Salesperson, TSP brute force)

1.2 Python Reference Model & Copy Semantics

Visual Diagram: Memory Reference Aliasing vs Shallow Copy vs Deep Copy



1.3 Dynamic List Over-Allocation & Recursion Scope

List Growth: In CPython, list.append() operates in amortized O(1) time because the list over-allocates contiguous memory via new_allocated = size + (size >> 3) + (size < 9 ? 3 : 6). However, inserting at index 0 (list.insert(0, x)) requires shifting all N elements in memory, yielding O(N) cost!

Scope in Backtracking: In recursive DFS, primitive variables require the nonlocal keyword to be reassigned in nested functions. Alternatively, pass state inside a mutable container (e.g. best = [float('inf')]) which is modified in-place across recursive call frames.

CHAPTER 2 - ARRAYS, STRINGS, FREQUENCY MAPS & HASHING

Arrays provide O(1) random index access. Hashing transforms complex objects into canonical signatures to enable O(1) grouping and retrieval.

2.1 Anagram Grouping by Canonical Frequency Signatures (LeetCode 49)

The Signature Trap: Sorting each word costs O(K log K) where K is string length. For N words of length K, total time is O(N * K log K). A superior approach constructs a 26-element character frequency tuple in O(K) time, achieving optimal **O(N * K)** total runtime.

Code: Optimal Anagram Grouping O(N * K)

```
from collections import defaultdict

def groupAnagrams(strs: list[str]) -> list[list[str]]:
    groups = defaultdict(list)
    for s in strs:
        # Build 26-char frequency tuple as immutable hashable key
        count = [0] * 26
        for ch in s:
            count[ord(ch) - ord('a')] += 1
        groups[tuple(count)].append(s)
    return list(groups.values())
```

Time: O(N * K)	Space: O(N * K)	Notes: Optimal hash map bucketing via 26-element count tuples.

2.2 Product of Array Except Self (Prefix & Suffix Running Products)

Constraint: Solve in O(N) time without using the division operator, and in O(1) auxiliary space (excluding output array).
Invariant: res[i] equals the product of all elements to the left of i multiplied by the product of all elements to the right of i.

Code: Product of Array Except Self (O(1) Auxiliary Space)

```
def productExceptSelf(nums: list[int]) -> list[int]:
    n = len(nums)
    res = [1] * n
    # Pass 1: Accumulate prefix products
    prefix = 1
    for i in range(n):
        res[i] = prefix
        prefix *= nums[i]
    # Pass 2: Accumulate suffix products into result in-place
    suffix = 1
    for i in range(n - 1, -1, -1):
        res[i] *= suffix
        suffix *= nums[i]
    return res
```

2.3 Virtual 1D Coordinate Mapping for 2D Matrices

Flattening an M x N matrix requires O(M * N) memory. Instead, compute virtual 2D coordinates from a 1D index idx in O(1) time: row = idx // cols and col = idx % cols. Conversely, convert 2D coordinates back: idx = r * cols + c.

CHAPTER 3 - TWO POINTERS: CONVERGING & READ/WRITE ARCHETYPES

The Two Pointers technique converts nested O(N^2) loops into a single linear O(N) sweep by maintaining directional invariants over iterable sequences.

3.1 The 4 Core Two-Pointer Archetypes

Archetype	Pointer Motion	Standard Interview Problems
1. Converging	Left starts at 0, Right starts at N-1, move inward	Two Sum II (Sorted), Container With Most Water, 3Sum, Palindrome
2. Read / Write	Both start at 0; Fast reads, Slow writes in-place	Remove Duplicates (LC 26), Move Zeroes (LC 283), Remove Element
3. Center Expand	Both start at center (i, i) or (i, i+1), expand outward	Longest Palindromic Substring (LC 5), Palindromic Substrings (LC 647)
4. Two-List Merge	One pointer on list A, one on list B, step smaller	Merge Sorted Arrays (LC 88), Intersection of Two Arrays (LC 350)

3.2 LeetCode 167: Two Sum II (Sorted Input Array)

Visual Diagram: Monotonic Decision Tree: Sorted Array Two Pointers

```
CONVERGING POINTER DECISION INVARIANT:
  Array: [ 2 , 7 , 11 , 15 ], Target = 9
           ^              ^
         Left           Right    ==> Sum = 2 + 15 = 17 > 9 (Too big! Decrement Right)

  Array: [ 2 , 7 , 11 , 15 ]
           ^        ^
         Left    Right          ==> Sum = 2 + 11 = 13 > 9 (Too big! Decrement Right)

  Array: [ 2 , 7 , 11 , 15 ]
           ^   ^
         Left Right             ==> Sum = 2 + 7 = 9 == 9 (MATCH FOUND!)
```

3.3 Read/Write Pointer Compacting (LeetCode 26: Remove Duplicates)

Code: In-Place Read/Write Array Compacting

```python
def removeDuplicates(nums: list[int]) -> int:
    if not nums: return 0
    write = 1  # Invariant: nums[:write] contains all unique elements seen
    for read in range(1, len(nums)):
        if nums[read] != nums[read - 1]:
            nums[write] = nums[read]
            write += 1
    return write  # Returns length of unique compact array
```

3.4 LeetCode 680: Valid Palindrome II (Greedy Branching)

Invariant: Advance left and right inward while characters match. Upon the first mismatch, test if deleting s[left] OR deleting s[right] yields a palindrome. No backtracking is needed because at most one deletion is permitted!

Code: Valid Palindrome II (Greedy Branch)

```python
def validPalindrome(s: str) -> bool:
    def check(l: int, r: int) -> bool:
        while l < r:
            if s[l] != s[r]: return False
            l += 1; r -= 1
        return True

    left, right = 0, len(s) - 1
    while left < right:
        if s[left] != s[right]:
            return check(left + 1, right) or check(left, right - 1)
        left += 1; right -= 1
    return True
```

CHAPTER 4 - FAST & SLOW POINTERS (FLOYD'S ALGORITHM)

Floyd's Cycle-Finding Algorithm uses two pointers advancing at different speeds (1 step vs 2 steps) to detect cycles and identify landmarks in O(1) space.

4.1 Mathematical Proof of Meeting in a Cycle

Visual Diagram: Floyd's Cycle Collision Proof

```
CYCLE GEOMETRY & MATHEMATICAL DERIVATION:
      Head -----> (Entrance) -------------> [ Meeting Point M ]
           <-- L -->         <------ d ----->
                             <---------- Cycle Length C ---------->

  1. Let distance from Head to Cycle Entrance = L.
  2. Let distance from Entrance to Meeting Point M = d.
  3. When Slow reaches M, it has traveled: Distance_Slow = L + d.
  4. Fast travels at 2x speed: Distance_Fast = 2 * (L + d).
  5. Fast must also satisfy: Distance_Fast = L + d + k * C (where k is number of full loops).
  6. Equating: 2*(L + d) = L + d + k*C  ===>  L + d = k*C  ===>  L = k*C - d!

  CRUCIAL CONCLUSION:
  Distance from Head to Entrance (L) equals distance from Meeting Point to Entrance (k*C - d)!
  Therefore, resetting Pointer 1 to Head and keeping Pointer 2 at M, then moving BOTH at 1 step/iter,
  guarantees they will collide EXACTLY AT THE ENTRANCE NODE after L steps!
```

4.2 LeetCode 142: Linked List Cycle II (Detect Cycle Entrance)

Code: Floyd's Cycle Entrance Detection

```python
def detectCycle(head):
    slow = fast = head
    has_cycle = False
    # Phase 1: Detect cycle existence
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
        if slow == fast:
            has_cycle = True
            break
    if not has_cycle: return None

    # Phase 2: Find cycle entrance node (Proof: L = k*C - d)
    p1 = head
    p2 = slow
    while p1 != p2:
        p1 = p1.next
        p2 = p2.next
    return p1  # Exactly at cycle entrance!
```

Time: O(N)	Space: O(1)	Notes: Guaranteed collision in < 2*N steps with zero extra memory.

4.3 LeetCode 287: Find the Duplicate Number (Array as Linked List)

Array-as-Graph Mapping: Given an array of size N + 1 with integers in [1..N], interpret index i as a node with directed edge to nums[i]. Because a duplicate value has multiple incoming edges, it forms the entrance to a directed cycle! Apply Floyd's Cycle Algorithm directly on array indices without modifying the array.

Code: Find Duplicate Number via Cycle Detection

```python
def findDuplicate(nums: list[int]) -> int:
    slow = fast = nums[0]
    while True:
        slow = nums[slow]
        fast = nums[nums[fast]]
        if slow == fast: break
    ptr1 = nums[0]
    ptr2 = slow
    while ptr1 != ptr2:
        ptr1 = nums[ptr1]
        ptr2 = nums[ptr2]
    return ptr1
```

CHAPTER 5 - SLIDING WINDOW PATTERN (FIXED & DYNAMIC)

The Sliding Window pattern converts contiguous range problems from O(N^2) to O(N) by tracking running state inside an active window [left..right].

5.1 Fixed-Size Sliding Window: Rolling Sum & Max Average (LC 643)

Visual Diagram: Fixed-Size Window Rolling Update

```
FIXED SLIDING WINDOW (Size k = 3):
  Array: [ 2, 1, 5, 1, 3, 2 ]
  Window 1: [ 2 , 1 , 5 ] , 1 , 3 , 2   Sum = 8
  Window 2:   2 , [ 1 , 5 , 1 ] , 3 , 2   Sum = 8 - 2 + 1 = 7  (Add right, subtract left!)
  Window 3:   2 , 1 , [ 5 , 1 , 3 ] , 2   Sum = 7 - 1 + 3 = 9  (Max so far!)
  Window 4:   2 , 1 , 5 , [ 1 , 3 , 2 ]   Sum = 9 - 5 + 2 = 6
```

Code: Fixed Window Rolling Average O(N)

```
def findMaxAverage(nums: list[int], k: int) -> float:
    curr_sum = sum(nums[:k])
    max_sum = curr_sum
    for i in range(k, len(nums)):
        curr_sum += nums[i] - nums[i - k]  # O(1) rolling update
        max_sum = max(max_sum, curr_sum)
    return max_sum / k
```

5.2 Dynamic Variable-Size Window: Longest Substring Without Repeating (LC 3)

Invariant: Expand right pointer and record character indices in a hash map. When duplicate character ch is encountered, jump left directly past its last recorded position: left = max(left, last_seen[ch] + 1).

Code: Longest Substring with Index Jump

```
def lengthOfLongestSubstring(s: str) -> int:
    last_seen = {}
    max_len = left = 0
    for right, ch in enumerate(s):
        if ch in last_seen and last_seen[ch] >= left:
            left = last_seen[ch] + 1  # Fast jump past duplicate!
        last_seen[ch] = right
        max_len = max(max_len, right - left + 1)
    return max_len
```

5.3 Dynamic Minimum Window: Minimum Size Subarray Sum (LC 209)

Invariant: Expand right until window sum >= target. Then greedily shrink left while sum >= target to minimize window length.

Code: Minimum Size Subarray Sum (O(N))

```
def minSubArrayLen(target: int, nums: list[int]) -> int:
    left = curr_sum = 0
    min_len = float('inf')
    for right in range(len(nums)):
        curr_sum += nums[right]
        while curr_sum >= target:
            min_len = min(min_len, right - left + 1)
            curr_sum -= nums[left]
            left += 1
    return min_len if min_len != float('inf') else 0
```

CHAPTER 6 - ADVANCED WINDOW & MONOTONIC DEQUE MAXIMUM

Advanced sliding window techniques manage complex state conditions and maintain extremum elements in strict linear time.

6.1 Character Replacement with Max Frequency Invariant (LeetCode 424)

Problem: Given string s and integer k, return the maximum length of a substring with identical letters after replacing at most k characters.
Invariant: A window is valid if: (window_length - max_frequency_char) <= k. We never need to shrink max_freq when sliding left because a smaller frequency cannot yield a longer valid window!

Code: Longest Repeating Character Replacement O(N)

```
def characterReplacement(s: str, k: int) -> int:
    counts = {}
    left = max_freq = 0
    for right in range(len(s)):
        counts[s[right]] = counts.get(s[right], 0) + 1
        max_freq = max(max_freq, counts[s[right]])
        # If replacements needed exceed k, slide left by 1
        if (right - left + 1) - max_freq > k:
            counts[s[left]] -= 1
            left += 1
    return len(s) - left
```

6.2 Monotonic Deque Sliding Window Maximum (LeetCode 239)

Problem: Find the maximum value in every sliding window of size K in strict O(N) time.
Invariant: Maintain a deque of indices in strictly decreasing order of their array values: nums[q[0]] > nums[q[1]] > ... The current window maximum is ALWAYS at q[0]!

Visual Diagram: Monotonic Deque Eviction & Extremum Tracking

```
MONOTONIC DEQUE STRUCTURAL STATE (nums = [1, 3, -1, -3, 5, 3, 6, 7], k = 3):
  i=0: val= 1  Deque: [0] (val: 1)
  i=1: val= 3  Pop 0 (1 <= 3). Deque: [1] (val: 3)
  i=2: val=-1  Push 2. Deque: [1, 2] (vals: 3, -1) ==> Window 1 Max = nums[q[0]] = 3
  i=3: val=-3  Push 3. Deque: [1, 2, 3] (vals: 3, -1, -3) ==> Window 2 Max = 3
  i=4: val= 5  Pop 1 (expired). Pop 3, 2 (-3,-1 <= 5). Deque: [4] (val: 5) ==> Max = 5
```

Code: Sliding Window Maximum (Monotonic Deque O(N))

```
from collections import deque

def maxSlidingWindow(nums: list[int], k: int) -> list[int]:
    q = deque()  # Stores indices in strictly decreasing value order
    res = []
    for i, num in enumerate(nums):
        # 1. Remove indices that fell out of current window
        if q and q[0] <= i - k:
            q.popleft()
        # 2. Maintain monotonic decreasing order: pop smaller values
        while q and nums[q[-1]] <= num:
            q.pop()
        q.append(i)
        # 3. Append current max once window reaches size k
        if i >= k - 1:
            res.append(nums[q[0]])
    return res
```

Time: O(N)	Space: O(K)	Notes: Each index is pushed and popped from deque at most once.

CHAPTER 7 - PREFIX SUMS, SUFFIXES & RANGE MANIPULATION

Prefix transformations precalculate cumulative properties to answer arbitrary subarray queries in O(1) time.

7.1 Subarray Sum Equals K (Negative Values Handling - LC 560)

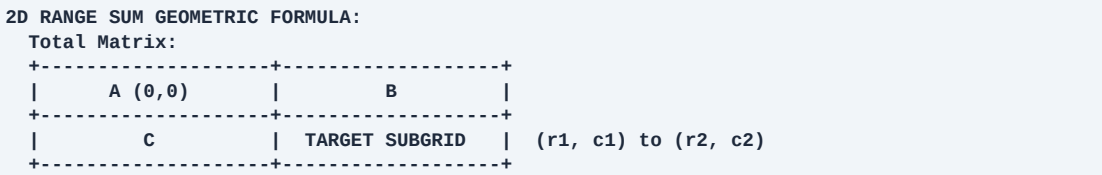
Mathematical Formula: If prefix_sum[R] - prefix_sum[L - 1] = K, then prefix_sum[L - 1] = prefix_sum[R] - K.
Why Sliding Window Fails: Sliding window requires monotonic growth (positive numbers only). If negative numbers exist, shrinking left or expanding right has unpredictable effects on sum. Only Prefix Sum + Hash Map works!

Code: Subarray Sum Equals K (O(N) Time & Space)

```
def subarraySum(nums: list[int], k: int) -> int:
    prefix_counts = {0: 1}  # Base case: empty subarray sum = 0
    curr_sum = 0
    count = 0
    for num in nums:
        curr_sum += num
        # If curr_sum - k was seen, valid subarrays end at current index
        if (curr_sum - k) in prefix_counts:
            count += prefix_counts[curr_sum - k]
        prefix_counts[curr_sum] = prefix_counts.get(curr_sum, 0) + 1
    return count
```

7.2 2D Matrix Range Sum Query (Prefix Sum Matrix - LC 304)

Visual Diagram: 2D Inclusion-Exclusion Principle



Code: 2D Prefix Sum Query O(1)

```
class NumMatrix:
    def __init__(self, matrix: list[list[int]]):
        if not matrix or not matrix[0]: return
        R, C = len(matrix), len(matrix[0])
        self.p = [[0] * (C + 1) for _ in range(R + 1)]
        for r in range(R):
            for c in range(C):
                self.p[r + 1][c + 1] = (matrix[r][c] + self.p[r][c + 1] +
                                        self.p[r + 1][c] - self.p[r][c])

    def sumRegion(self, r1: int, c1: int, r2: int, c2: int) -> int:
        return (self.p[r2 + 1][c2 + 1] - self.p[r1][c2 + 1] -
                self.p[r2 + 1][c1] + self.p[r1][c1])
```

CHAPTER 8 - STACKS & MONOTONIC STACK ARCHITECTURE

A Monotonic Stack maintains elements in strictly ascending or descending order to resolve 'Next Greater/Smaller Element' queries in O(N) time.

8.1 Daily Temperatures & Next Greater Element (LeetCode 739)

Invariant: Maintain a stack of indices with decreasing temperatures. When encountering temperature T[i] greater than T[stack[-1]], pop the index and record the day distance: res[prev] = i - prev.

Code: Daily Temperatures (Monotonic Stack O(N))

```
def dailyTemperatures(temperatures: list[int]) -> list[int]:
    n = len(temperatures)
    res = [0] * n
    stack = []  # Stores indices in monotonic decreasing temperature order
    for i, t in enumerate(temperatures):
        while stack and temperatures[stack[-1]] < t:
            prev_i = stack.pop()
            res[prev_i] = i - prev_i
        stack.append(i)
    return res
```

8.2 MinStack with O(1) Minimum Retrieval (LeetCode 155)

Parallel Stack Invariant: Maintain an auxiliary stack min_stack where min_stack[-1] holds the minimum value of all elements currently on the main stack.

Code: MinStack with Parallel Minimum Stack

```
class MinStack:
    def __init__(self):
        self.stack = []
        self.min_stack = []

    def push(self, val: int) -> None:
        self.stack.append(val)
        curr_min = min(val, self.min_stack[-1] if self.min_stack else val)
        self.min_stack.append(curr_min)

    def pop(self) -> None:
        self.stack.pop()
        self.min_stack.pop()

    def top(self) -> int: return self.stack[-1]
    def getMin(self) -> int: return self.min_stack[-1]
```

8.3 Score of Parentheses (LeetCode 856)

Rule: () scores 1; AB scores A + B; (A) scores 2 * A. Use a stack to accumulate scores at the current nesting depth.

Code: Score of Parentheses

```
def scoreOfParentheses(s: str) -> int:
    stack = [0]
    for ch in s:
        if ch == '(':
            stack.append(0)
        else:
            v = stack.pop()
            stack[-1] += max(2 * v, 1)
    return stack.pop()
```

CHAPTER 9 - QUEUES & DEQUE ARCHITECTURE

Queues guarantee FIFO ordering. In Python, always use collections.deque for queues to achieve O(1) front operations.

9.1 collections.deque Chunked Memory vs list.pop(0) O(N) Degradation

Visual Diagram: Chunked Doubly-Linked Memory vs Contiguous Array

```
MEMORY ARCHITECTURE COMPARISON:
  1. Python standard list (Contiguous Array):
     [ Item 0 | Item 1 | Item 2 | Item 3 | Item 4 ]
     list.pop(0) removes Item 0 ===> MUST SHIFT ALL N ELEMENTS LEFT IN MEMORY! (Cost: O(N))

  2. Python collections.deque (Doubly-Linked 64-Element Chunks):
     [ Chunk A: 64 ptrs ] <---> [ Chunk B: 64 ptrs ] <---> [ Chunk C: 64 ptrs ]
     popleft() simply advances head pointer inside Chunk A; when empty, deallocates chunk in O(1)!
```

9.2 LeetCode 933: Number of Recent Calls (Sliding Time Eviction Window)

Problem: Count calls within the past 3000 ms. Evict expired timestamps from front of deque in amortized O(1) time.

Code: Recent Counter with Sliding Deque Eviction

```
from collections import deque

class RecentCounter:
    def __init__(self):
        self.q = deque()

    def ping(self, t: int) -> int:
        self.q.append(t)
        # Evict timestamps older than t - 3000
        while self.q and self.q[0] < t - 3000:
            self.q.popleft()
        return len(self.q)
```

9.3 LeetCode 232: Implement Queue Using Two Stacks

Amortized Analysis: in_stack receives pushes. out_stack serves pops. When out_stack is empty, transfer all elements from in_stack to out_stack, reversing their order into true FIFO. Each element moves at most once into and out of each stack -> **Amortized O(1) pop**.

Code: Queue using 2 Stacks (Amortized O(1))

```
class MyQueue:
    def __init__(self):
        self.in_stack = []
        self.out_stack = []

    def push(self, x: int) -> None:
        self.in_stack.append(x)

    def pop(self) -> int:
        self.peek()
        return self.out_stack.pop()

    def peek(self) -> int:
        if not self.out_stack:
            while self.in_stack:
                self.out_stack.append(self.in_stack.pop())
        return self.out_stack[-1]

    def empty(self) -> bool:
        return not self.in_stack and not self.out_stack
```

CHAPTER 10 - BINARY SEARCH & SEARCH-ON-ANSWER

Binary search applies to any monotonic search space where a condition transitions from False to True exactly once.

10.1 Standard Closed-Interval Binary Search & Lower Bound

Code: Lower Bound Template

```
# Lower Bound: First index where nums[i] >= target (bisect_left equivalent)
def lower_bound(nums: list[int], target: int) -> int:
    left, right = 0, len(nums)  # Half-open interval [0, n)
    while left < right:
        mid = left + (right - left) // 2
        if nums[mid] < target:
            left = mid + 1
        else:
            right = mid
    return left  # Returns len(nums) if all elements < target
```

10.2 LeetCode 74: Search a 2D Matrix (Virtual 1D Coordinates)

An M x N matrix where each row is sorted and begins after the previous row ends is equivalent to a 1D sorted array of length M * N. Map 1D index mid to 2D coordinates: r = mid // N, c = mid % N.

Code: Search 2D Matrix (O(log(M * N)))

```
def searchMatrix(matrix: list[list[int]], target: int) -> bool:
    if not matrix or not matrix[0]: return False
    m, n = len(matrix), len(matrix[0])
    left, right = 0, m * n - 1
    while left <= right:
        mid = left + (right - left) // 2
        val = matrix[mid // n][mid % n]
        if val == target: return True
        elif val < target: left = mid + 1
        else: right = mid - 1
    return False
```

10.3 LeetCode 875: Koko Eating Bananas (Search-on-Answer)

Monotonicity Invariant: If eating speed K allows Koko to finish within H hours, any speed > K will also finish within H. Binary search on speed in range [1, max(piles)].

Code: Koko Eating Bananas (Search-on-Answer)

```
import math

def minEatingSpeed(piles: list[int], h: int) -> int:
    left, right = 1, max(piles)
    ans = right
    while left <= right:
        speed = left + (right - left) // 2
        hours = sum(math.ceil(p / speed) for p in piles)
        if hours <= h:
            ans = speed      # Valid speed, try to find smaller
            right = speed - 1
        else:
            left = speed + 1 # Too slow, must increase speed
    return ans
```

CHAPTER 11 - LINKED LISTS MANIPULATION

11.1 Reversing a Linked List In-Place

Visual Diagram: Iterative Reversal Invariant

```
POINTER REWIRING STATE TRANSITION:
  Original: None <--- [prev]      [curr] ---> [nxt] ---> ...
  Action:   curr.next = prev
  Advance:  prev = curr; curr = nxt
```

Code: Iterative In-Place Reversal

```
def reverseList(head):
    prev, curr = None, head
    while curr:
        nxt = curr.next
        curr.next = prev
        prev = curr
        curr = nxt
    return prev
```

11.2 Merge Two Sorted Lists (Dummy Sentinel Pattern)

Code: Merge Two Sorted Lists

```
def mergeTwoLists(l1, l2):
    dummy = ListNode(0)
    tail = dummy
    while l1 and l2:
        if l1.val <= l2.val:
            tail.next = l1
            l1 = l1.next
        else:
            tail.next = l2
            l2 = l2.next
        tail = tail.next
    tail.next = l1 if l1 else l2
    return dummy.next
```

11.3 LeetCode 143: Reorder List (3-Stage In-Place Pipeline)

Pipeline: (1) Find middle via fast & slow pointers -> (2) Reverse second half in-place -> (3) Interleave two halves.

Code: Reorder List (3-Stage Pipeline)

```
def reorderList(head) -> None:
    if not head or not head.next: return
    # Stage 1: Find middle node
    slow, fast = head, head.next
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
    # Stage 2: Reverse second half
    second = slow.next
    slow.next = None
    prev = None
    while second:
        nxt = second.next
        second.next = prev
        prev = second
        second = nxt
    # Stage 3: Interleave first and second halves
    first, second = head, prev
    while second:
        tmp1, tmp2 = first.next, second.next
        first.next = second
        second.next = tmp1
        first, second = tmp1, tmp2
```

CHAPTER 12 - BINARY TREES & BINARY SEARCH TREES

12.1 Level Order BFS Traversal with Queue Batching

Code: Level Order Traversal BFS

```
from collections import deque

def levelOrder(root) -> list[list[int]]:
    if not root: return []
    res = []
    q = deque([root])
    while q:
        level_size = len(q)  # Freeze size of current level
        level = []
        for _ in range(level_size):
            node = q.popleft()
            level.append(node.val)
            if node.left: q.append(node.left)
            if node.right: q.append(node.right)
        res.append(level)
    return res
```

12.2 Balanced Binary Tree (Bottom-Up O(N) Depth Check)

A naive top-down check recomputes height at each node yielding O(N^2). A bottom-up approach returns height from subtrees or -1 immediately upon detecting an imbalance, running in optimal **O(N)**.

Code: Balanced Binary Tree (Bottom-Up O(N))

```
def isBalanced(root) -> bool:
    def check(node):
        if not node: return 0
        left = check(node.left)
        if left == -1: return -1
        right = check(node.right)
        if right == -1: return -1
        if abs(left - right) > 1: return -1
        return 1 + max(left, right)
    return check(root) != -1
```

12.3 Validate Binary Search Tree via Range Invariant (LeetCode 98)

Invariant: Every node must satisfy low < node.val < high. Propagate bounds downward across recursive calls.

Code: Validate BST Range Bounds

```
def isValidBST(root) -> bool:
    def validate(node, low=float('-inf'), high=float('inf')):
        if not node: return True
        if not (low < node.val < high): return False
        return (validate(node.left, low, node.val) and
                validate(node.right, node.val, high))
    return validate(root)
```

12.4 Lowest Common Ancestor (LCA)

Code: Lowest Common Ancestor

```
def lowestCommonAncestor(root, p, q):
    if not root or root == p or root == q: return root
    left = lowestCommonAncestor(root.left, p, q)
    right = lowestCommonAncestor(root.right, p, q)
    if left and right: return root  # Split point: root is LCA!
    return left if left else right
```

CHAPTER 13 - HEAPS & PRIORITY QUEUES

13.1 Building a Heap: heapify O(N) vs Repeated Push O(N log N)

Mathematical Proof: In a complete tree of height H, the majority of nodes reside near leaves (height 0 or 1). Sifting down from bottom to top takes: Sum(h=0 to log N) [(N / 2^(h+1)) * O(h)] = O(N). Never push elements one-by-one (O(N log N)); always use heapq.heapify() for O(N) initialization!

13.2 Top-K Frequent Elements (Min-Heap of Size K)

Code: Top-K Frequent Elements O(N log K)

```
from collections import Counter
import heapq

def topKFrequent(nums: list[int], k: int) -> list[int]:
    counts = Counter(nums)
    # Maintain a Min-Heap of size K: (frequency, num)
    heap = []
    for num, freq in counts.items():
        heapq.heappush(heap, (freq, num))
        if len(heap) > k:
            heapq.heappop(heap)  # Evict lowest frequency
    return [num for freq, num in heap]
```

Time: O(N log K)	Space: O(N)	Notes: Keeps heap bounded to K elements rather than full N.

13.3 Find Median from Data Stream (Dual-Heap Architecture - LC 295)

Visual Diagram: Dual-Heap Equilibrium

```
DUAL-HEAP BALANCE INVARIANT:
  Lower Half: Max-Heap (stores -val)  |  Upper Half: Min-Heap (stores val)
  [ elements <= median ]               |  [ elements >= median ]
  Invariant 1: len(max_heap) == len(min_heap) OR len(max_heap) == len(min_heap) + 1
  Invariant 2: Every element in max_heap <= Every element in min_heap
```

Code: Dual-Heap Median Finder (O(log N) Add, O(1) Query)

```
class MedianFinder:
    def __init__(self):
        self.small = []  # Max-Heap (stores -val)
        self.large = []  # Min-Heap (stores val)

    def addNum(self, num: int) -> None:
        # Push to small (max-heap), pop max and push to large (min-heap)
        val = -heapq.heappushpop(self.small, -num)
        heapq.heappush(self.large, val)
        # Rebalance sizes: small can have at most 1 more element than large
        if len(self.large) > len(self.small):
            heapq.heappush(self.small, -heapq.heappop(self.large))

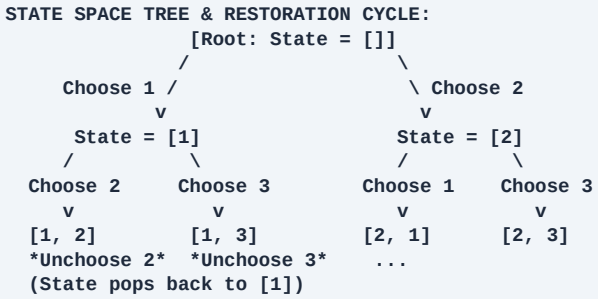
    def findMedian(self) -> float:
        if len(self.small) > len(self.large):
            return float(-self.small[0])
        return (-self.small[0] + self.large[0]) / 2.0
```

CHAPTER 14 - RECURSION & BACKTRACKING MASTERCLASS

Backtracking systematically searches state spaces by incrementally building candidates and abandoning branches as soon as constraints are violated.

14.1 The Universal Backtracking Lifecycle: Choose -> Explore -> Unchoose

Visual Diagram: Backtracking Tree Navigation



14.2 Decision Rules: Subsets vs Combinations vs Permutations

Problem Type	Recursion Index Step	Loop Structure	Duplicate Handling
Subsets	dfs(i + 1)	for j in range(i, n)	Sort array; skip if j > i and nums[j]==nums[j-1]
Combinations	dfs(i) (reuse allowed)	for j in range(i, n)	Sort candidates to prune early when target < 0
Permutations	dfs() (no start index)	for j in range(0, n)	Maintain used boolean set/array

14.3 LeetCode 90: Subsets II (Handling Duplicate Elements)

Code: Subsets II with Duplicate Pruning

```
def subsetsWithDup(nums: list[int]) -> list[list[int]]:
    nums.sort()  # Sort to bring duplicates adjacent
    res = []
    def dfs(start, path):
        res.append(list(path))
        for i in range(start, len(nums)):
            # Skip duplicate values at the same recursion depth
            if i > start and nums[i] == nums[i - 1]:
                continue
            path.append(nums[i])      # Choose
            dfs(i + 1, path)          # Explore
            path.pop()                # Unchoose
    dfs(0, [])
    return res
```

14.4 LeetCode 22: Generate Parentheses (Catalan Invariant)

Code: Generate Parentheses (Valid Balance)

```
def generateParenthesis(n: int) -> list[str]:
    res = []
    def dfs(open_c, close_c, path):
        if len(path) == 2 * n:
            res.append("".join(path))
            return
        if open_c < n:
            path.append('(')
            dfs(open_c + 1, close_c, path)
            path.pop()
        if close_c < open_c:
            path.append(')')
            dfs(open_c, close_c + 1, path)
            path.pop()
    dfs(0, 0, [])
    return res
```

CHAPTER 15 - GRAPHS & GRID ALGORITHMS

15.1 Number of Islands (In-Place Sinking BFS - LC 200)

Code: Number of Islands (In-Place BFS Sinking)

```
def numIslands(grid: list[list[str]]) -> int:
    if not grid: return 0
    rows, cols = len(grid), len(grid[0])
    islands = 0

    def bfs(r, c):
        q = deque([(r, c)])
        grid[r][c] = "0"  # Sink island in-place to avoid extra visited set!
        while q:
            cr, cc = q.popleft()
            for dr, dc in [(-1, 0), (1, 0), (0, -1), (0, 1)]:
                nr, nc = cr + dr, cc + dc
                if 0 <= nr < rows and 0 <= nc < cols and grid[nr][nc] == "1":
                    grid[nr][nc] = "0"
                    q.append((nr, nc))

    for r in range(rows):
        for c in range(cols):
            if grid[r][c] == "1":
                bfs(r, c)
                islands += 1
    return islands
```

15.2 Topological Sort: Kahn's In-Degree Algorithm (Course Schedule)

Code: Kahn's Topological Sort BFS

```
def canFinish(numCourses: int, prerequisites: list[list[int]]) -> bool:
    adj = {i: [] for i in range(numCourses)}
    in_degree = [0] * numCourses
    for crs, pre in prerequisites:
        adj[pre].append(crs)
        in_degree[crs] += 1
    q = deque([i for i in range(numCourses) if in_degree[i] == 0])
    processed = 0
    while q:
        curr = q.popleft()
        processed += 1
        for nxt in adj[curr]:
            in_degree[nxt] -= 1
            if in_degree[nxt] == 0:
                q.append(nxt)
    return processed == numCourses  # True if DAG has no cycles!
```

15.3 Dijkstra's Shortest Path Algorithm (Min-Heap Priority Queue)

Code: Dijkstra's Algorithm (Min-Heap O((V + E) log V))

```
import heapq

def dijkstra(graph: dict, start: int) -> dict:
    distances = {node: float('inf') for node in graph}
    distances[start] = 0
    pq = [(0, start)]  # (distance, node)
    while pq:
        curr_d, curr_u = heapq.heappop(pq)
        if curr_d > distances[curr_u]:
            continue  # Outdated lazy heap entry
        for neighbor, weight in graph[curr_u]:
            d = curr_d + weight
            if d < distances[neighbor]:
                distances[neighbor] = d
                heapq.heappush(pq, (d, neighbor))
    return distances
```

CHAPTER 16 - DYNAMIC PROGRAMMING: 1D & STREAK PARADIGMS

Dynamic Programming solves optimization problems by combining solutions to overlapping subproblems with optimal substructure.

16.1 Kadane's Algorithm & Streak Optimization (Maximum Subarray)

Streak Invariant: At each element x, decide whether to append x to the running positive streak OR start a new streak starting at x: curr_max = max(x, curr_max + x).

Code: Kadane's Algorithm (O(N) Time, O(1) Space)

```
def maxSubArray(nums: list[int]) -> int:
    curr_max = global_max = nums[0]
    for x in nums[1:]:
        curr_max = max(x, curr_max + x)  # Either extend streak or restart
        global_max = max(global_max, curr_max)
    return global_max
```

16.2 House Robber (1D State Space Reduction - LC 198)

Recurrence: dp[i] = max(nums[i] + dp[i-2], dp[i-1]). Notice we only ever need the previous two values: rob1 and rob2. Optimize from O(N) space down to O(1) space!

Code: House Robber (O(1) Space)

```
def rob(nums: list[int]) -> int:
    rob1 = rob2 = 0
    for n in nums:
        temp = max(n + rob1, rob2)
        rob1 = rob2
        rob2 = temp
    return rob2
```

16.3 LeetCode 322: Coin Change (Unbounded Knapsack)

State: dp[a] = minimum coins required to make amount a.
Recurrence: dp[a] = min(dp[a], 1 + dp[a - c]) for all c <= a.

Code: Coin Change (Bottom-Up Tabulation)

```
def coinChange(coins: list[int], amount: int) -> int:
    dp = [float('inf')] * (amount + 1)
    dp[0] = 0
    for a in range(1, amount + 1):
        for c in coins:
            if a - c >= 0:
                dp[a] = min(dp[a], 1 + dp[a - c])
    return dp[amount] if dp[amount] != float('inf') else -1
```

CHAPTER 17 - DYNAMIC PROGRAMMING: 2D, KNAPSACK & LIS

17.1 0/1 Knapsack Space Optimization (Reverse Inner Loop)

Crucial Invariant: When reducing a 2D knapsack DP table to a 1D array of capacity W, traverse the weight loop in **REVERSE** (from W down to wt). This guarantees that an item is used at most once in each state calculation!

Code: 0/1 Knapsack (1D Reverse Loop Space Optimization)

```
def knapsack01(weights: list[int], values: list[int], W: int) -> int:
    dp = [0] * (W + 1)
    for wt, val in zip(weights, values):
        # Traverse backwards to prevent multiple inclusion of same item!
        for w in range(W, wt - 1, -1):
            dp[w] = max(dp[w], val + dp[w - wt])
    return dp[W]
```

17.2 Longest Increasing Subsequence (Patience Sort O(N log N) - LC 300)

Standard DP runs in O(N^2). By maintaining an array tails where tails[i] stores the smallest tail of all increasing subsequences of length i+1, we can use binary search (bisect_left) to achieve optimal **O(N log N)**.

Code: LIS via Binary Search (Patience Sort O(N log N))

```
from bisect import bisect_left

def lengthOfLIS(nums: list[int]) -> int:
    tails = []
    for x in nums:
        idx = bisect_left(tails, x)
        if idx == len(tails):
            tails.append(x)
        else:
            tails[idx] = x  # Greedily lower the tail of length idx+1
    return len(tails)
```

17.3 Longest Common Subsequence (2D Grid DP - LC 1143)

Recurrence: If text1[i] == text2[j], dp[i][j] = 1 + dp[i-1][j-1]. Otherwise, take max(dp[i-1][j], dp[i][j-1]).

Code: Longest Common Subsequence O(M * N)

```
def longestCommonSubsequence(text1: str, text2: str) -> int:
    m, n = len(text1), len(text2)
    dp = [[0] * (n + 1) for _ in range(m + 1)]
    for i in range(m):
        for j in range(n):
            if text1[i] == text2[j]:
                dp[i + 1][j + 1] = 1 + dp[i][j]
            else:
                dp[i + 1][j + 1] = max(dp[i][j + 1], dp[i + 1][j])
    return dp[m][n]
```

CHAPTER 18 - GREEDY ALGORITHMS & INTERVAL SCHEDULING

18.1 Jump Game I & II (Reachability Horizon & Implicit BFS)

Jump Game I: Track max_reach. If current index i > max_reach, we are stuck.
Jump Game II: Maintain active jump window [curr_end, next_reach]. When reaching curr_end, advance jump counter.

Code: Jump Game I & II (Greedy Linear O(N))

```
def canJump(nums: list[int]) -> bool:
    max_reach = 0
    for i, jump in enumerate(nums):
        if i > max_reach: return False
        max_reach = max(max_reach, i + jump)
    return True

def jump(nums: list[int]) -> int:
    jumps = curr_end = next_reach = 0
    for i in range(len(nums) - 1):
        next_reach = max(next_reach, i + nums[i])
        if i == curr_end:
            jumps += 1
            curr_end = next_reach
    return jumps
```

18.2 Merge Overlapping Intervals (LeetCode 56)

Sorting Invariant: Sort intervals by start time: intervals.sort(key=lambda x: x[0]). Iterate forward: if current start <= previous end, merge by updating end: res[-1][1] = max(res[-1][1], curr[1]).

Code: Merge Intervals O(N log N)

```
def merge(intervals: list[list[int]]) -> list[list[int]]:
    intervals.sort(key=lambda x: x[0])
    merged = []
    for iv in intervals:
        if not merged or merged[-1][1] < iv[0]:
            merged.append(iv)
        else:
            merged[-1][1] = max(merged[-1][1], iv[1])
    return merged
```

18.3 Gas Station (Circular Circuit Deficit - LC 134)

Code: Gas Station Circular Deficit O(N)

```
def canCompleteCircuit(gas: list[int], cost: list[int]) -> int:
    if sum(gas) < sum(cost): return -1  # Impossible to complete circuit
    total_tank = curr_tank = start = 0
    for i in range(len(gas)):
        diff = gas[i] - cost[i]
        curr_tank += diff
        if curr_tank < 0:
            start = i + 1  # Cannot start anywhere from prior start to i
            curr_tank = 0
    return start
```

CHAPTER 19 - BITWISE, TRIES & NUMBER THEORY

19.1 Implement Trie (Prefix Tree - LeetCode 208)

Code: Trie Prefix Tree Implementation

```
class TrieNode:
    def __init__(self):
        self.children = {}
        self.is_end = False

class Trie:
    def __init__(self):
        self.root = TrieNode()

    def insert(self, word: str) -> None:
        curr = self.root
        for ch in word:
            if ch not in curr.children:
                curr.children[ch] = TrieNode()
            curr = curr.children[ch]
        curr.is_end = True

    def search(self, word: str) -> bool:
        curr = self.root
        for ch in word:
            if ch not in curr.children: return False
            curr = curr.children[ch]
        return curr.is_end

    def startsWith(self, prefix: str) -> bool:
        curr = self.root
        for ch in prefix:
            if ch not in curr.children: return False
            curr = curr.children[ch]
        return True
```

19.2 Essential Bitwise Hacks

Operation	Bitwise Expression	Mathematical Mechanism
Clear lowest set bit	n & (n - 1)	Clears lowest 1 bit (Kernighan's count set bits)
Extract lowest set bit	n & (-n)	Isolates rightmost 1 bit via two's complement
Power of two test	n > 0 and (n & (n-1)) == 0	Powers of two have exactly one set bit
XOR Self Cancellation	x ^ x == 0, x ^ 0 == x	Finds unique element among pairs (LC 136)

19.3 Number Theory: Euclidean GCD & Prime Check

Code: Euclidean GCD & Optimized O(sqrt(N)) Prime Check

```
def gcd(a: int, b: int) -> int:
    while b:
        a, b = b, a % b
    return a

def is_prime(n: int) -> bool:
    if n < 2: return False
    if n in (2, 3): return True
    if n % 2 == 0 or n % 3 == 0: return False
    i = 5
    while i * i <= n:
        if n % i == 0 or n % (i + 2) == 0: return False
        i += 6
    return True
```

CHAPTER 20 - PATTERN-SELECTION GUIDE & FALSE FRIENDS

20.1 Input Constraints vs Big-O Target Decision Matrix

Input Size N	Allowable Big-O	Likely Candidate Algorithms & Paradigms
N <= 10 - 12	O(N!) or O(N^2 * 2^N)	Permutations, Hamiltonian Path, Traveling Salesperson
N <= 20 - 25	O(2^N) or O(N * 2^N)	Subsets backtracking, Bitmask DP, Meet-in-the-middle
N <= 500	O(N^3)	Floyd-Warshall all-pairs shortest paths, Matrix chain mult
N <= 5,000	O(N^2)	2D Grid DP, all-pairs comparison, Bubble/Insertion sort
N <= 10^5 - 10^6	O(N log N) or O(N)	Sorting, Monotonic Stack, Two Pointers, Sliding Window, Heap
N >= 10^7 - 10^9	O(log N) or O(1)	Binary Search on answer, Euclidean GCD, Bitwise math formulas

20.2 The 'False Friends' Guide: Debunking Common Traps

Symptom / Scenario	The Trap (Wrong Choice)	The Correct Mechanism
Subarray sum with negatives	Sliding Window (Fails: non-monotonic)	Prefix Sum + Hash Map (LC 560)
Shortest path, unweighted	Dijkstra (Heap adds O(log V) overhead)	Standard Queue BFS (Runs in O(V + E))
Cycle in directed graph	Union-Find / DSU (Cannot orient edges)	Kahn's BFS or 3-Color DFS
Running median stream	Binary search insertion in list O(N)	Dual-Heap: Max-Heap + Min-Heap O(log N)
Two Sum on sorted array	Hash Map lookup (Consumes O(N) space)	Two Pointers converging (O(1) space)

20.3 5-Phase Technical Interview Problem-Solving Protocol

Phase 1: Clarify & Constrain: Read inputs, constraints, edge cases (empty list, duplicates, negatives).
Phase 2: Establish Invariant & Complexity: State the target Big-O explicitly before writing code.
Phase 3: Write Clean, Modular Code: Follow standard boilerplate patterns; use descriptive pointer names.
Phase 4: Trace Dry-Run: Walk through a small test case manually line-by-line.
Phase 5: Check Off-by-One & Bounds: Verify loop termination, 0-indexing, and boundary conditions.

Tested, bug-free implementations ready for instant deployment during technical interviews.

Template 1: Binary Search Variants (Exact, Lower Bound, Upper Bound)

Code: Master Binary Search Templates

```
# 1. Closed-Interval Exact Search: [left, right]
def binary_search(nums: list[int], target: int) -> int:
    left, right = 0, len(nums) - 1
    while left <= right:
        mid = left + (right - left) // 2
        if nums[mid] == target: return mid
        elif nums[mid] < target: left = mid + 1
        else: right = mid - 1
    return -1

# 2. Lower Bound: First index where nums[i] >= target (bisect_left)
def lower_bound(nums: list[int], target: int) -> int:
    left, right = 0, len(nums)
    while left < right:
        mid = left + (right - left) // 2
        if nums[mid] < target: left = mid + 1
        else: right = mid
    return left

# 3. Upper Bound: First index where nums[i] > target (bisect_right)
def upper_bound(nums: list[int], target: int) -> int:
    left, right = 0, len(nums)
    while left < right:
        mid = left + (right - left) // 2
        if nums[mid] <= target: left = mid + 1
        else: right = mid
    return left
```

Template 2: Monotonic Stack (Next Greater Element Template)

Code: Master Monotonic Stack Template

```
def next_greater_elements(nums: list[int]) -> list[int]:
    n = len(nums)
    res = [-1] * n
    stack = []  # Stores indices of elements waiting for next greater
    for i, num in enumerate(nums):
        while stack and nums[stack[-1]] < num:
            prev_i = stack.pop()
            res[prev_i] = num
        stack.append(i)
    return res
```

Template 3: Disjoint Set Union (DSU with Path Compression & Rank)

Code: Disjoint Set Union (DSU / Union-Find)

```
class DSU:
    def __init__(self, n: int):
        self.parent = list(range(n))
        self.rank = [1] * n
        self.num_components = n

    def find(self, i: int) -> int:
        if self.parent[i] == i:
            return i
        self.parent[i] = self.find(self.parent[i])  # Path compression!
        return self.parent[i]

    def union(self, i: int, j: int) -> bool:
        root_i, root_j = self.find(i), self.find(j)
        if root_i == root_j:
            return False  # Already in same set; cycle detected!
        # Union by rank: attach smaller tree under root of larger tree
        if self.rank[root_i] < self.rank[root_j]:
            root_i, root_j = root_j, root_i
        self.parent[root_j] = root_i
        if self.rank[root_i] == self.rank[root_j]:
            self.rank[root_i] += 1
        self.num_components -= 1
        return True
```

Template 4: Dual-Heap Running Median Template

Code: Running Median Finder

```
import heapq

class RunningMedian:
    def __init__(self):
        self.small = []  # Max-Heap (stores negative values)
        self.large = []  # Min-Heap (stores positive values)

    def add(self, num: int) -> None:
        val = -heapq.heappushpop(self.small, -num)
        heapq.heappush(self.large, val)
        if len(self.large) > len(self.small):
            heapq.heappush(self.small, -heapq.heappop(self.large))

    def get_median(self) -> float:
        if len(self.small) > len(self.large):
            return float(-self.small[0])
        return (-self.small[0] + self.large[0]) / 2.0
```

The authoritative quick-reference table of operations and interview sanity checks.

Data Structure Operations Complexity Matrix

Data Structure	Access	Search	Insertion	Deletion	Space
Array / List	O(1)	O(N)	O(N) (O(1) append)	O(N)	O(N)
Deque	O(N)	O(N)	O(1) both ends	O(1) both ends	O(N)
Hash Map / Set	N/A	O(1) avg / O(N)	O(1) avg / O(N)	O(1) avg / O(N)	O(N)
Binary Heap	O(1) min/max	O(N)	O(log N)	O(log N) pop	O(N)
Balanced BST	O(log N)	O(log N)	O(log N)	O(log N)	O(N)
Trie (Prefix)	N/A	O(L) word len	O(L)	O(L)	O(N * L)
Graph (Adj List)	N/A	O(V + E) BFS/DFS	O(1) add edge	O(deg(V))	O(V + E)

Sorting Algorithms Complexity Reference

Algorithm	Best Time	Average Time	Worst Time	Space	Stable?
TimSort (Python)	O(N)	O(N log N)	O(N log N)	O(N)	Yes
QuickSort	O(N log N)	O(N log N)	O(N^2)	O(log N)	No
MergeSort	O(N log N)	O(N log N)	O(N log N)	O(N)	Yes
HeapSort	O(N log N)	O(N log N)	O(N log N)	O(1)	No

THE 60-SECOND PRE-CODING SANITY CHECKLIST

1. **Empty & 1-Element Inputs:** Does your algorithm immediately guard against len(nums) == 0 or 1?
2. **Off-by-One Boundaries:** Are your intervals strictly closed [L, R] with <= or half-open [L, R) with <?
3. **Negative & Duplicate Values:** Does prefix sum account for negative values? Does backtracking skip duplicate branches?
4. **Reference Mutability:** Did you alias a list or modify grid coordinates without tracking visited state?
5. **Big-O Compliance:** Does your solution fit cleanly within the 10^7 operations-per-second budget?